You said:

1. **Approach and tradeoffs**

Currently, each request runs two counts over ~1M rows. The cost is paid per request, hence the CPU hitting 85% at ~200 requests per min. The fix would be moving it to a per refresh cost.

Freshness target: Standing and percentile may lag by about 5 minutes. The user's own CSR is always live. Age is measured from when the snapshot was captured, so a slow refresh can push it slightly past 5 minutes.

Design: A Supabase cron job refreshes two materialized views together, in one job,every 5 minutes:

- A score view (scope, score, users_at_or_above) holding, for each scope (global, continent), the count of eligible users at or above each totalRating, plus a total eligible-user count per scope with CSR > 0. Eligibility (XP, status), and timezone-to-continent mapping are applied at refresh time
- A per-user snapshot view: one row per eligible user holding their continent and the score they had at the last refresh. Having a row means the user was included in the snapshot.

Each view has its own unique index, which REFRESH ... CONCURRENTLY requires, so refreshes don't block SELECTs (refresh CPU and I/O can still slow reads).

Request flow:

1. Read the live CSR from AverageRanks (one-row lookup)
2. In the scoreview, find the smallest score strictly above the live CSR and read its users_at_or_above. If nothing, usersAbove = 0
3. Read the user's snapshot row. If the user was in the queried scope's snapshot and their snapshot score is greater than the live CSR (their score dropped since the refresh), subtract 1, because their old self is counted above them.
4. Standing = usersAbove +1, percentile = usersAbove * 100 / total

'Above' Means strictly greater, matching the original query, so tied users share a standing.

Every user's question is the same ("how many eligible users are above rating X?"), so we compute the ranking once per refresh and serve every request from it. Each request is a few indexed lookups that grow logarithmically with the number of distinct scores, avoiding the scan over every user above the requester.

Tradeoffs:

- Rank can lag other users' activity. Users can verify their own score against a workout, but not their position among thousands of others.
- With floating-point CSR values, the number of distinct scores may approach the number of eligible users. Refreshing the cumulative score distribution and per-user snapshots every five minutes therefore introduces significant background aggregation and index-maintenance costs. I'd benchmark refresh duration, CPU and storage against production-sized data before rollout.
- Near-ties at the top of the board can be briefly stale. If required, a live top-N path could be added.

The existing percentile divides by the global total even for continent scope. I'd preserve this for now and flag it as a likely bug.The supplied code doesn't filter by friends, so I preserve that behavior and flag it. If friends-only ranking is intended, I'd implement it as a live query restricted to friend IDs, since it's per user.

2. **Rollout and verification**

- **Staging:** build the objects, unique index, and cron job on production-sized data, measuring refresh time and read latency. Confirm if avg latency < 200ms.
- **Correctness check:** verify the new path against an exact COUNT over the same snapshot, covering ties, score drops, continent membership, missing users, and empty scopes.

- **Shadow mode** behind a feature flag: for a sample of requests, run both paths, return the old result, and log differences. Pass condition: differences are explained by snapshot age, with no systematic bias.
- **Ramp** 5% -> 25% -> 50% -> 100%, advancing only if latency and DB CPU are in thresholds. The flag provides instant rollback.
- **Cleanup:** Remove the old path after one clean release cycle

3. **Post-launch monitoring and risks**

Metrics: endpoint p50/p95 latency (target avg <200ms), DB CPU at peak (baseline 85%), snapshot age, refresh duration and failures, and sampled standing error versus a live count.

Risks:

- **Refresh fails silently and data goes stale:** Alert if snapshot age exceeds 15 minutes; retry, and fall back to the live query via the flag.
- **Refresh slows as users grow:** Alert when refresh duration exceeds ~20% of the interval (or 1 minute). Lengthen the interval, bucket the rating so the refresh is a GROUP BY over a smaller output, or move to a Redis sorted set.

```
model User {
  id               String              @id @default(cuid())
  experiencePoints  Int                 @default(0)
  timeZone         String              @default("UTC")
  status           String              @default("-") // "-" = active
  averageRanks      AverageRanks[]
  exercisePeakRanks ExercisePeakRanking[]
}

model AverageRanks {
  id          String @id @default(cuid())
  userId      String
  user        User   @relation(fields: [userId], references: [id])
```

```
  totalRating Float  @default(0) // Combined Skill Rating (CSR)
  season      Int

  @@unique([userId, season])
  @@index([totalRating, userId])
}

model ExercisePeakRanking {
  id             String @id @default(cuid())
  userId         String
  exerciseId     String
  exerciseRating Int    @default(0)
  user           User   @relation(fields: [userId], references: [id])
}\

// packages/api/src/router/user.ts

getSelfForLeaderboardByMainRank: protectedProcedure
    .input(
        z.object({
            scope: z.enum(["global", "continent", "friends"]),
            continent: z.string().optional(),
        })
    )
    .query(async ({ ctx, input }) => {
        const { scope, continent } = input;
        const user = await ctx.prisma.user.findUniqueOrThrow({
            where: { id: ctx.userId },
            select: {
                timeZone: true,
                experiencePoints: true,
                averageRanks: { select: { totalRating: true } },
            },
        });
        const userSeasonRanks = user.averageRanks[0];
        if (!userSeasonRanks) return null;
        const usersAbove = await ctx.prisma.user.count({
            where: {
```

```
experiencePoints: { gte: MIN_LEADERBOARD_EXP },
          status: "-",
          ...(scope === "continent" && continent
            ? { timeZone: { in: getTimezonesForContinent(continent) } }
            : {}),
          averageRanks: {
          some: {
            totalRating: { gt: userSeasonRanks.totalRating },
          },yup
        },
},
});
const totalUsers = await ctx.prisma.user.count({
    where: {
        experiencePoints: { gte: MIN_LEADERBOARD_EXP },
        status: "-",
        averageRanks: { some: { totalRating: { gt: 0 } } },
    },
});
return {
    standing: usersAbove + 1,
    percentile: totalUsers === 0 ? 0 : (usersAbove * 100) / totalUsers,
    data: { csr: userSeasonRanks.totalRating },
};
}),\
```

what would the updated schema and user.ts look like?

Worked for 1m 54s

ChatGPT said:

Your existing `User` and `AverageRanks` Prisma models can stay largely unchanged. The main additions are **two PostgreSQL materialized views, their indexes, and a scheduled refresh job**.

I'd implement this using a SQL migration and Prisma's `$queryRaw` for the endpoint. Prisma doesn't currently manage PostgreSQL materialized views directly through `model` declarations.

Prisma Docu... +1

1. Updated schema

`schema.prisma`

The only new Prisma model you need is a timezone-to-continent lookup table. This keeps the mapping consistent between your TypeScript application and the SQL refresh.

```
model User {
  id                String                @id @default(cuid())
  experiencePoints  Int                   @default(0)
  timeZone          String                @default("UTC")
  status            String                @default("-")
  averageRanks      AverageRanks[]
  exercisePeakRanks ExercisePeakRanking[]
}

model AverageRanks {
  id          String @id @default(cuid())
  userId      String
  user        User   @relation(fields: [userId], references: [id])
  totalRating Float  @default(0)
  season      Int

  @@unique([userId, season])
  @@index([totalRating, userId])
}

model ExercisePeakRanking {
  id             String @id @default(cuid())
  userId         String
  exerciseId     String
  exerciseRating Int    @default(0)
  user           User   @relation(fields: [userId], references: [id])
}

model TimeZoneContinent {
  timeZone  String @id
```

```
  continent String

  @@index([continent])
}
```

Populate `TimeZoneContinent` using the same mapping as `getTimezonesForContinent()`.

One important detail: the original query uses `averageRanks: { some: ... }`, which considers **any season**, whereas the live CSR comes from `averageRanks[0]`.

In the implementation below, I use `MAX(totalRating)` across a user's seasons for the ranking population. This preserves the original `some` predicate for counts, assuming the ranking population should include all seasons. A season-specific leaderboard would instead require an explicit season filter.

2. SQL migration — materialized views

Create a Prisma migration with:

```
npx prisma migrate dev --create-only \
  --name leaderboard_snapshots
```

Then add the SQL below to its `migration.sql`.

A. Per-user snapshot

This stores one row for each eligible user, including their captured score and continent.

```
CREATE MATERIALIZED VIEW "LeaderboardUserSnapshot" AS
SELECT
    u.id AS "userId",
    MAX(ar."totalRating") AS score,
    tz.continent
FROM "User" u
JOIN "AverageRanks" ar
    ON ar."userId" = u.id
LEFT JOIN "TimeZoneContinent" tz
    ON tz."timeZone" = u."timeZone"
WHERE
```

```
    u."experiencePoints" >= 8650
    AND u.status = '-'
GROUP BY u.id, tz.continent;

CREATE UNIQUE INDEX leaderboard_user_snapshot_pk
ON "LeaderboardUserSnapshot" ("userId");
```

Here, `8650` represents `MIN_LEADERBOARD_EXP`; use your actual constant when deploying the migration.

B. Score distribution snapshot

The second view aggregates eligible users by score and scope, then calculates the cumulative count.

```
CREATE MATERIALIZED VIEW "LeaderboardScoreSnapshot" AS
WITH scoped_scores AS (
    -- Global scope
    SELECT
        'global'::text AS scope,
        'all'::text AS "scopeKey",
        score
    FROM "LeaderboardUserSnapshot"

    UNION ALL

    -- Continent scope
    SELECT
        'continent'::text AS scope,
        continent AS "scopeKey",
        score
    FROM "LeaderboardUserSnapshot"
    WHERE continent IS NOT NULL
),
score_counts AS (
    SELECT
        scope,
        "scopeKey",
        score,
        COUNT(*)::bigint AS count
    FROM scoped_scores
```

```
    GROUP BY scope, "scopeKey", score
)
SELECT
    scope,
    "scopeKey",
    score,

    SUM(count) OVER (
        PARTITION BY scope, "scopeKey"
        ORDER BY score DESC
        ROWS BETWEEN UNBOUNDED PRECEDING
                 AND CURRENT ROW
    )::bigint AS "usersAtOrAbove",

    SUM(count) FILTER (WHERE score > 0) OVER (
        PARTITION BY scope, "scopeKey"
    )::bigint AS "totalEligibleUsers"

FROM score_counts;

CREATE UNIQUE INDEX leaderboard_score_snapshot_pk
ON "LeaderboardScoreSnapshot" (
    scope, "scopeKey", score
);

CREATE INDEX leaderboard_score_lookup_idx
ON "LeaderboardScoreSnapshot" (
    scope, "scopeKey", score
)
INCLUDE ("usersAtOrAbove");
```

The final index is designed for the endpoint's lookup: find the smallest score strictly above the user's live CSR.

Notice that `"totalEligibleUsers"` is a per-scope denominator. Your current endpoint instead uses the global denominator even for continent scope, so the TypeScript implementation below deliberately reads the global total.

Important: this view depends on the per-user materialized view. Refreshing them sequentially is not an atomic operation. That matters for the self-correction logic, which I'll address after the code.

3. Updated `user.ts`

This replaces the two expensive Prisma counts with one indexed SQL query against the snapshots.

```
type LeaderboardSnapshotResult = {
  usersAbove: bigint;
  totalUsers: bigint;
};

getSelfForLeaderboardByMainRank: protectedProcedure
  .input(
    z.object({
      scope: z.enum(["global", "continent", "friends"]),
      continent: z.string().optional(),
    })
  )
  .query(async ({ ctx, input }) => {
    const { scope, continent } = input;

    // 1. Fetch the user's LIVE rating
    const user = await ctx.prisma.user.findUniqueOrThrow({
      where: { id: ctx.userId },
      select: {
        timeZone: true,
        experiencePoints: true,
        averageRanks: {
          select: { totalRating: true },
        },
      },
    });

    const userSeasonRanks = user.averageRanks[0];

    if (!userSeasonRanks) return null;

    const liveCSR = userSeasonRanks.totalRating;

    // Preserve existing behavior:
    // "friends" and continent without a value use global.
    const isContinent =
      scope === "continent" && Boolean(continent);
```

```
const scopeName = isContinent ? "continent" : "global";
const scopeKey = isContinent ? continent! : "all";

// 2. Read precomputed ranking + user snapshot
const [result] = await ctx.prisma.$queryRaw<
  LeaderboardSnapshotResult[]
>`
  SELECT
    GREATEST(
      COALESCE((
        SELECT s."usersAtOrAbove"
        FROM "LeaderboardScoreSnapshot" s
        WHERE s.scope = ${scopeName}
          AND s."scopeKey" = ${scopeKey}
          AND s.score > ${liveCSR}
        ORDER BY s.score ASC
        LIMIT 1
      ), 0)
      -
      CASE
        WHEN EXISTS (
          SELECT 1
          FROM "LeaderboardUserSnapshot" us
          WHERE us."userId" = ${ctx.userId}
            AND us.score > ${liveCSR}
            AND (
              ${scopeName} = 'global'
              OR us.continent = ${scopeKey}
            )
        )
        THEN 1
        ELSE 0
      END,
      0
    )::bigint AS "usersAbove",

    COALESCE((
      SELECT s."totalEligibleUsers"
      FROM "LeaderboardScoreSnapshot" s
      WHERE s.scope = 'global'
        AND s."scopeKey" = 'all'
```

```
        LIMIT 1
      ), 0)::bigint AS "totalUsers"
  `;

  const usersAbove = Number(result.usersAbove);
  const totalUsers = Number(result.totalUsers);

  return {
    standing: usersAbove + 1,
    percentile:
      totalUsers === 0
        ? 0
        : (usersAbove * 100) / totalUsers,
    data: {
      csr: liveCSR,
    },
  };
}),
```

How the new query works

Suppose your cached score distribution contains:

Score	Users at or above
100	1
90	3
80	7
70	12

If the current user's live CSR is `85`:

1. Find the smallest cached score greater than 85: `90`.
2. Return its cumulative count: `3`.
3. Check whether the user's cached score was above 85 and belonged to the requested scope.
4. Subtract one if their previous score would otherwise count themselves.

This is an indexed lookup rather than counting eligible users individually on every request.

PostgreSQL raw queries return BIGINT values as JavaScript `bigint`, which is why the example converts them to numbers before calculating the response.

4. Supabase cron job

Use a database function to refresh the two views in the correct order within a single database transaction.

```
CREATE OR REPLACE FUNCTION refresh_leaderboard_snapshots()
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY
        "LeaderboardUserSnapshot";

    REFRESH MATERIALIZED VIEW CONCURRENTLY
        "LeaderboardScoreSnapshot";
END;
$$;
```

Then schedule it:

```
SELECT cron.schedule(
    'refresh-leaderboard',
    '*/5 * * * *',
    $$SELECT refresh_leaderboard_snapshots();$$
);
```

This uses Supabase's PostgreSQL cron functionality. Supabase Do... +1

Why use one function?

PostgreSQL can execute both refreshes within the function's transaction, so readers don't see the new user snapshot paired with an old score distribution between commits.

`CONCURRENTLY` also allows normal reads while each materialized view refreshes, provided the view is populated and has an appropriate unique index. PostgreSQL +1

You should still prevent overlapping scheduled jobs and monitor for failures. At a million rows, a five-minute refresh may be expensive, particularly because the score distribution can have nearly a million distinct floating-point values.

5. Important details before shipping

There are four things I'd address before using this as your take-home implementation.

1. Add a snapshot timestamp.

Include the following field in the `SELECT` defining `LeaderboardUserSnapshot`:

```
transaction_timestamp() AS "capturedAt"
```

Because refreshes run in a single transaction, you can read this timestamp to measure snapshot age and detect stale refreshes. For your 15-minute alert, track this rather than relying on when cron last started.

2. Decide how seasons should work.

Your original Prisma query checks whether *any* `AverageRanks` record exceeds the threshold. The implementation above preserves that counting population by using each user's maximum rating, but the requesting user's CSR still comes from `averageRanks[0]`.

For a production implementation, I would explicitly select the current season and filter the snapshots to that season. That would eliminate ambiguity and make the live CSR and snapshot scores directly comparable.

3. Understand the self-correction difference.

Your original query can count the requesting user as being above themselves when another season has a higher rating. The self-correction intentionally excludes them in this situation, so it slightly changes that edge case. That's generally preferable for leaderboard semantics, but should be documented.

4. Benchmark refresh costs before committing to this architecture.

Your endpoint becomes much cheaper, but the refresh still needs to aggregate approximately one million user scores, calculate cumulative totals, and maintain two materialized views.

I'd test the following:

Metric	Target
Endpoint average latency	Below 200 ms
Refresh duration	Ideally below 60 s
Refresh interval	5 minutes
Snapshot age alert	Over 15 minutes
DB CPU	Significantly below 85% at 200 req/min
Rank correctness	Exact against the same snapshot

There is also a subtle dependency: `LeaderboardScoreSnapshot` is derived from `LeaderboardUserSnapshot`. If the refresh fails partway through, the transaction should roll back both refreshes, leaving the previous consistent pair available.

For the take-home: I'd include the Prisma model, SQL materialized view definitions, unique indexes, refreshed endpoint, and cron function as the implementation deliverable. The main thing I'd emphasize in the proposal is that you're replacing two expensive counts per request with a small number of indexed lookups, while explicitly accepting bounded staleness and paying the aggregation cost on a schedule instead.

You said:

```
// 2. Read precomputed ranking + user snapshot
const [result] = await ctx.prisma.$queryRaw<
  LeaderboardSnapshotResult[]
>`
  SELECT
    GREATEST(
      COALESCE((
        SELECT s."usersAtOrAbove"
        FROM "LeaderboardScoreSnapshot" s
        WHERE s.scope = ${scopeName}
          AND s."scopeKey" = ${scopeKey}
          AND s.score > ${liveCSR}
        ORDER BY s.score ASC
```

```
          LIMIT 1
        ), 0)
        -
        CASE
          WHEN EXISTS (
            SELECT 1
            FROM "LeaderboardUserSnapshot" us
            WHERE us."userId" = ${ctx.userId}
              AND us.score > ${liveCSR}
              AND (
                ${scopeName} = 'global'
                OR us.continent = ${scopeKey}
              )
          )
          THEN 1
          ELSE 0
        END,
        0
      )::bigint AS "usersAbove",

      COALESCE((
        SELECT s."totalEligibleUsers"
        FROM "LeaderboardScoreSnapshot" s
        WHERE s.scope = 'global'
          AND s."scopeKey" = 'all'
        LIMIT 1
      ), 0)::bigint AS "totalUsers"
  `;
```

Does no prisma method exist for this

ChatGPT said:

Yes. **Prisma does have methods for performing these lookups**, but there's a catch: Prisma doesn't natively expose PostgreSQL materialized views through its schema in the same way it exposes regular tables.

You have three options:

1. `$queryRaw` **(current approach):** Directly query the materialized views using SQL. This is the most straightforward for your design.
2. **Prisma Client queries:** Define the materialized views as read-only models using `@@map`, then use `findFirst()`, `findUnique()`, and `count()` where appropriate.
3. **Prisma** `view` **definitions:** Prisma supports database views through its `view` feature, but materialized views are different from ordinary SQL views, so support depends on the Prisma version and how you manage the database schema.

For your take-home, option 2 is possible and would make the endpoint more consistent with the existing codebase.

What Prisma Client would look like

If you represent your materialized views as Prisma `view` blocks, you can query them through Prisma Client. This is a read-only mapping; the SQL migration must still create and refresh the materialized views. Prisma's documentation does not officially support materialized views as a distinct schema feature. Prisma Docu... +1

For example, assuming your version supports the mapping:

```
generator client {
  provider        = "prisma-client-js"
  previewFeatures = ["views"]
}

view LeaderboardScoreSnapshot {
  scope              String
  scopeKey           String
  score              Float
  usersAtOrAbove     BigInt
  totalEligibleUsers BigInt
}

view LeaderboardUserSnapshot {
  userId    String
  score     Float
  continent String?
}
```

Your endpoint's lookup becomes:

```
const [scoreRow, userSnapshot, totalRow] =
  await Promise.all([
    ctx.prisma.leaderboardScoreSnapshot.findFirst({
      where: {
        scope: scopeName,
        scopeKey,
        score: { gt: liveCSR },
      },
      orderBy: { score: "asc" },
      select: { usersAtOrAbove: true },
    }),

    ctx.prisma.leaderboardUserSnapshot.findFirst({
      where: {
        userId: ctx.userId,
      },
    }),

    ctx.prisma.leaderboardScoreSnapshot.findFirst({
      where: {
        scope: "global",
        scopeKey: "all",
      },
      select: { totalEligibleUsers: true },
    }),
  ]);

let usersAbove = Number(scoreRow?.usersAtOrAbove ?? 0n);

if (
  userSnapshot &&
  userSnapshot.score > liveCSR &&
  (scopeName === "global" ||
    userSnapshot.continent === scopeKey)
) {
  usersAbove--;
}

usersAbove = Math.max(0, usersAbove);
```

```
const totalUsers = Number(
  totalRow?.totalEligibleUsers ?? 0n
);
```

This is much easier to read, although it involves three Prisma operations instead of one SQL statement.

There is one important tradeoff: the queries aren't guaranteed to read the same snapshot generation if a refresh commits between them. You'd need a read transaction with suitable isolation, or a versioned snapshot design, to guarantee consistency.

My recommendation: For the LiftOff take-home, I'd keep `$queryRaw` for this particular operation. Your complex SQL is isolated in one place, and a single statement reads a consistent PostgreSQL snapshot under normal `READ COMMITTED` isolation. It also demonstrates that you understand how to use PostgreSQL's indexed materialized views rather than relying entirely on the ORM.

You said:

why is order by score required

ChatGPT said:

`ORDER BY score ASC` is required because your materialized view stores **cumulative counts of users at or above each score**.

You need to find the **smallest score strictly greater than the user's live CSR** to get the correct count.

For example, your materialized view contains:

Score	usersAtOrAbove
100	1
90	3
80	7
70	12
60	20

Suppose the user's live CSR is **85**.

```
SELECT "usersAtOrAbove"
FROM "LeaderboardScoreSnapshot"
WHERE score > 85
ORDER BY score ASC
LIMIT 1;
```

The query:

1. `WHERE score > 85` finds scores 90 and 100.
2. `ORDER BY score ASC` puts 90 before 100.
3. `LIMIT 1` selects score 90, returning **3 users above**.

Without `ORDER BY`, PostgreSQL could return either score 90 or 100. If it returns 100, you'd incorrectly calculate only 1 user above.

Why ASC instead of DESC?

Because you're looking for the **next higher score**, not the highest score.

- `ASC` → nearest score greater than 85 → correct.
- `DESC` → highest score greater than 85 → incorrect.

Performance: Your index on `(scope, scopeKey, score)` supports this operation efficiently. PostgreSQL can seek to the first score greater than 85 and stop after finding one row, rather than sorting the entire materialized view.

This is what makes your lookup approximately O(log n) rather than scanning all eligible users.

You said:

isnt the indexing of the score distribution snapshot gonna have a massive impact

ChatGPT said:

Yes. **Indexing the score distribution snapshot could have a significant impact on refresh performance, especially with ~1 million users and floating-point CSR values.**

But there's an important distinction: **the index makes your reads cheaper while making refreshes more expensive.** Your design is deliberately trading background computation for faster requests.

1. The problem is the number of distinct scores

If your `totalRating` is a float, you could have almost as many distinct scores as users.

Scenario	Distinct scores	Score snapshot rows
Integer CSR (0–1,000)	≤1,001	≤~7,000 across 7 scopes
Floats rounded to 2 decimals (0–1,000)	≤100,001	≤~700,000
High-precision floats	~1,000,000	Up to ~2,000,000

The exact size depends on the score distribution and the number of continent scopes.

With high-precision scores, your supposedly aggregated score distribution could contain nearly twice as many rows as your user table because each user contributes to both the global and continent scopes.

That means PostgreSQL must calculate and maintain indexes on potentially millions of rows every five minutes.

2. What happens during each refresh?

With `REFRESH MATERIALIZED VIEW CONCURRENTLY`, PostgreSQL:

1. Executes the underlying aggregation query to generate a new result set.
2. Compares the new results with the existing materialized view.
3. Applies the differences while maintaining the indexes.

The important part is that **PostgreSQL still recomputes the full query**. `CONCURRENTLY` does not mean incremental aggregation.

It can also be slower than a non-concurrent refresh when many rows change. PostgreSQL

For your design, every change in a user's score can affect the cumulative counts for many score rows. So even if relatively few users change ratings, a large portion of the score distribution may change.

This makes your cumulative materialized view particularly expensive to refresh.

3. There's also a redundant index in the implementation

Previously, I suggested:

```
CREATE UNIQUE INDEX leaderboard_score_snapshot_pk
ON "LeaderboardScoreSnapshot" (
    scope, "scopeKey", score
);

CREATE INDEX leaderboard_score_lookup_idx
ON "LeaderboardScoreSnapshot" (
    scope, "scopeKey", score
)
INCLUDE ("usersAtOrAbove");
```

Both use the same three leading columns.

You can replace them with a single index:

```
CREATE UNIQUE INDEX leaderboard_score_snapshot_idx
ON "LeaderboardScoreSnapshot" (
    scope, "scopeKey", score
)
INCLUDE ("usersAtOrAbove", "totalEligibleUsers");
```

This satisfies the unique-index requirement for concurrent refreshes and supports your indexed score lookup. You avoid maintaining a second B-tree, although the additional included columns make the remaining index larger.

4. Would bucketing be better?

Potentially, yes.

Instead of storing every distinct score, you could group users into score ranges.

Illustrative snapshot size per scope

Hypothetical distribution: 1M distinct raw scores versus 1,000 fixed-width buckets. Logarithmic scale is not used.

However, a bucket's cumulative count alone cannot give an exact standing. You would still need to count users within the requester's bucket, ideally using a suitable index on the underlying user scores.

That gives you a different tradeoff:

Approach	Refresh cost	Request cost
Exact cumulative materialized view	Potentially high	Very low
Bucketed cumulative view + in-bucket count	Lower snapshot cost	Small live count
Original two counts	None	High

The bucketed approach is attractive because it avoids maintaining a cumulative result for almost every unique floating-point score.

5. What I would change in your proposal

I wouldn't immediately abandon the exact materialized view. Your original baseline is **85% CPU at roughly 200 requests/minute**, and that's the cost of repeated expensive counts.

Even an expensive refresh every five minutes could be a considerable improvement.

But I would benchmark the exact view against the bucketed alternative before committing.

A useful calculation is:

- Original system: approximately 1,000 requests per five minutes, each running two counts.
- Exact materialized view: two refreshes per five minutes, plus cheap indexed lookups.
- Bucketed approach: a smaller refresh every five minutes, plus indexed in-bucket counts per request.

The total CPU improvement depends on the refresh cost and the number of rows examined by the new queries, not just the reduction in query frequency.

One especially important point: Your materialized view's cumulative calculations may be more expensive than maintaining its index. The `SUM(...) OVER (ORDER BY score DESC)` must process the sorted score distribution, and changes can propagate through many cumulative rows.

So yes, indexing matters, but **the full recomputation and cumulative-count maintenance are probably the larger architectural concern**.

For a take-home, I'd present the exact materialized-view solution as the initial implementation, explicitly call out this scalability risk, and explain that bucketing is the next option to benchmark if refresh costs become problematic.

You said:

how do you allow reads to a materialized view during a refresh. how would you handle the 2 views such that the user snapshot is confirmed to be based on latest refresh

Worked for 8s

ChatGPT said:

You can solve both problems using `REFRESH MATERIALIZED VIEW CONCURRENTLY` **and a single database transaction**.

The important distinction is that `CONCURRENTLY` keeps reads available, while the transaction ensures both views become visible together.

1. How reads work during a refresh

PostgreSQL supports:

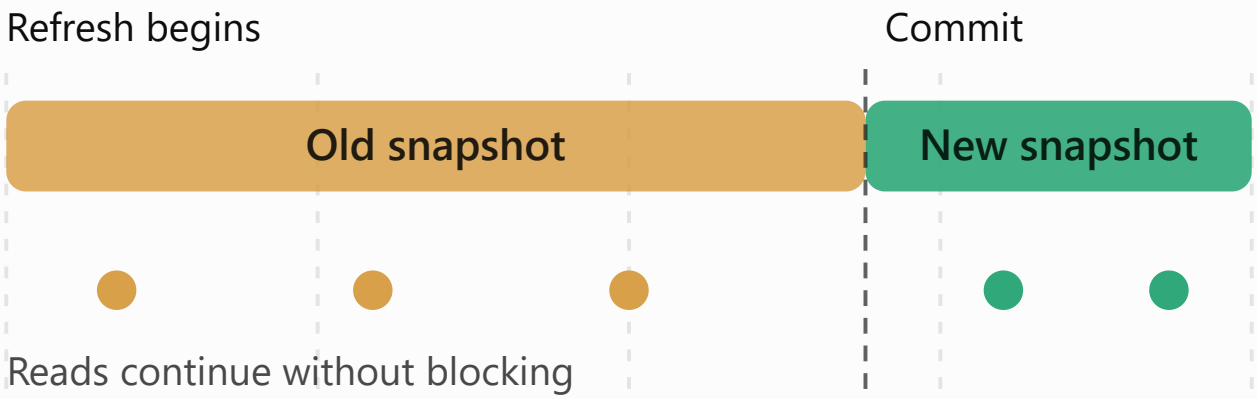
```
REFRESH MATERIALIZED VIEW CONCURRENTLY
    "LeaderboardUserSnapshot";
```

With a concurrent refresh:

1. PostgreSQL computes the new materialized-view results.
2. It compares them with the existing rows and applies the changes.
3. Normal `SELECT` queries continue reading committed data throughout the refresh.
4. Once the transaction commits, the updated results become visible to new queries.

Without `CONCURRENTLY`, the refresh takes a stronger lock and blocks ordinary reads until completion. PostgreSQL

Single materialized view refresh

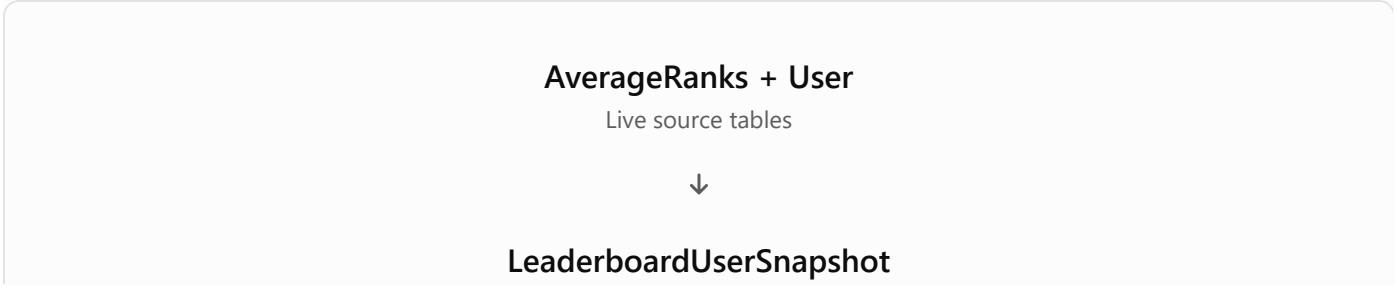


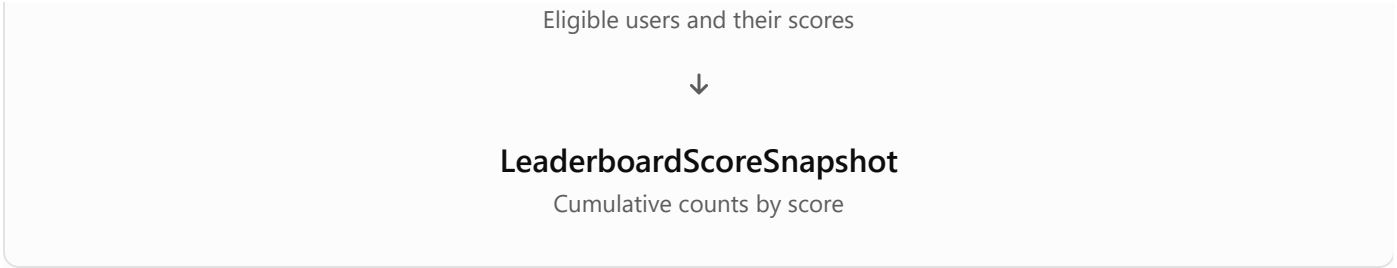
Conceptual visibility: queries continue using committed data while the refresh runs. A query's own database snapshot determines what it sees.

The prerequisite is a qualifying unique index on each populated materialized view.

2. How to keep your two materialized views consistent

Your second view depends on the first:





The problem with refreshing them independently is this:

- At 12:00, both views contain generation A.
- At 12:05, the user snapshot refreshes to B.
- At 12:05:30, a request reads the new user snapshot but the old score distribution.
- At 12:06, the score distribution finally refreshes to B.

That could cause incorrect self-correction, since the two views represent different generations.

Solution: Refresh both in one transaction

```
BEGIN;

REFRESH MATERIALIZED VIEW CONCURRENTLY
    "LeaderboardUserSnapshot";

REFRESH MATERIALIZED VIEW CONCURRENTLY
    "LeaderboardScoreSnapshot";

COMMIT;
```

This is supported in PostgreSQL: unlike `CREATE INDEX CONCURRENTLY`, `REFRESH MATERIALIZED VIEW CONCURRENTLY` can run inside a transaction. pgfence

Here's the visibility timeline:

Time	Background refresh	What API requests see
12:00	Both views at A	A + A
12:05	Start transaction	A + A
12:05:20	User snapshot refreshed	A + A

12:05:50	Score snapshot refreshed	A + A
12:05:51	`COMMIT`	B + B

The underlying view changes are invisible to other transactions until commit.

This guarantees that requests do not see the new user snapshot paired with the old score snapshot, assuming both are refreshed only through this coordinated process.

An additional subtlety: PostgreSQL's default `READ COMMITTED` isolation gives each SQL statement its own consistent snapshot. Since your `$queryRaw` reads both views within a single statement, it will see either generation A or B, not a mix.

3. How I'd implement the Supabase cron job

I would wrap the refreshes in a PostgreSQL function:

```
CREATE OR REPLACE FUNCTION refresh_leaderboard()
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
    REFRESH MATERIALIZED VIEW CONCURRENTLY
        "LeaderboardUserSnapshot";

    REFRESH MATERIALIZED VIEW CONCURRENTLY
        "LeaderboardScoreSnapshot";
END;
$$;
```

Schedule one job:

```
SELECT cron.schedule(
    'refresh-leaderboard',
    '*/5 * * * *',
    $$SELECT refresh_leaderboard();$$
);
```

A PostgreSQL function executes within its caller's transaction. If either refresh fails, the function's transaction rolls back, preventing a partial update.

You'd also want to serialize refresh runs and monitor cron failures.

4. Does this guarantee that the snapshot is the latest?

There are two different guarantees:

Consistency: Yes. Both materialized views can represent the same completed refresh generation.

Freshness: Not necessarily. They might both represent the last successful refresh from 10 minutes ago if subsequent jobs failed.

To monitor freshness, I'd maintain a small metadata table updated after both refreshes:

```
CREATE TABLE "LeaderboardRefreshState" (
    id integer PRIMARY KEY
        CHECK (id = 1),
    "lastCompletedAt" timestamptz NOT NULL
);
```

At the end of your refresh function:

```
INSERT INTO "LeaderboardRefreshState"
    (id, "lastCompletedAt")
VALUES (1, clock_timestamp())
ON CONFLICT (id) DO UPDATE
SET "lastCompletedAt" =
    EXCLUDED."lastCompletedAt";
```

This update belongs inside the same transaction, so it only becomes visible after both materialized views are successfully refreshed.

Then you can measure:

```
SELECT
    now() - "lastCompletedAt" AS snapshot_age
```

```
FROM "LeaderboardRefreshState"
WHERE id = 1;
```

One nuance: this records when the refresh completed, not exactly when the source data was captured. To measure true data age, also record the snapshot's capture time.

5. What about new user scores changing during the refresh?

Consider:

- 12:05:00 — refresh starts.
- 12:05:10 — another user finishes a workout.
- 12:05:30 — refresh completes.

Depending on when PostgreSQL's refresh statement captured its source-data snapshot, that workout might not appear in the newly refreshed views.

That's expected in your design: **the user snapshot and score distribution are internally consistent, but not necessarily current with every committed workout.**

This is acceptable under your five-minute staleness target, provided you account for refresh duration when measuring actual data age.

For your take-home, I would describe the solution as:

> Both materialized views are refreshed sequentially within a single transaction, using `REFRESH MATERIALIZED VIEW CONCURRENTLY` to keep reads available. The user snapshot is refreshed first, followed by its dependent score distribution. The transaction commits only after both succeed, ensuring requests never observe mismatched snapshot generations. Refresh completion and source snapshot age are monitored separately to detect stale data.

That is a strong explanation of both concurrency and consistency without adding unnecessary infrastructure.